

252-0027

Einführung in die Programmierung Übungen

Woche 2: Git und EBNF

Fabian Rohlik

Departement Informatik

ETH Zürich

Organisatorisches

- Mein Name: Fabian Rohlik
- Bei Fragen: frohlik@student.ethz.ch
- der Ordner u01 auf der Website der Vorlesung hatte einen Fehler

Notizen

- cs.rohlik.net, dann auf EProg Übungsstunde klicken

Heute

- EBNF Recap von letzter und dieser Woche
- Setup von Git
- EBNF Prüfungsaufgaben

EBNF Recap

- Syntax

EBNF Recap

- Syntax
- $\{[\langle \text{ausdruck} \rangle]\}$ und $[\{\langle \text{ausdruck} \rangle\}]$ ist gleich zu $\{\langle \text{ausdruck} \rangle\}$

EBNF Recap

`<digit>` $\leftarrow$ `0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9`

`<integer>` $\leftarrow$ `[ + | - ] <digit> {<digit>}`

Ableitungen

- wir wollen Korrektheit zeigen
- also „präsentieren“, warum eine bestimmte Zeichenfolge von einer EBNF-Grammatik erlaubt wird
- bisher haben wir uns das informell überlegt
- um es formell zu lösen, verwenden wir **Ableitungen**

Repetition: Ableitungen

- Ableitungsbaum
 - Wurzel ist Namen der Startregel
 - Blätter sind Zeichen
 - Verbindungen stehen für **einen** Ableitungsschritt

Beispiel: Ableitung von -127 als Baum

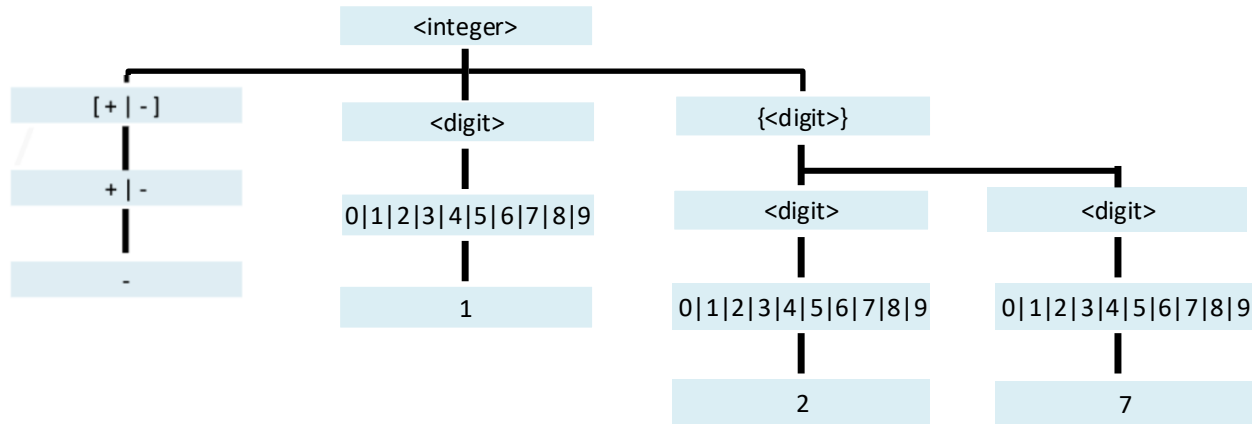
<digit> ← 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<integer> ← [+ | -] <digit> {<digit>}

Beispiel: Ableitung von -127 als Baum

$\langle \text{digit} \rangle \quad \leftarrow \quad 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

$\langle \text{integer} \rangle \quad \leftarrow \quad [+ \mid -] \langle \text{digit} \rangle \{ \langle \text{digit} \rangle \}$



Beispiel: Ableitung von -12.75 als Baum

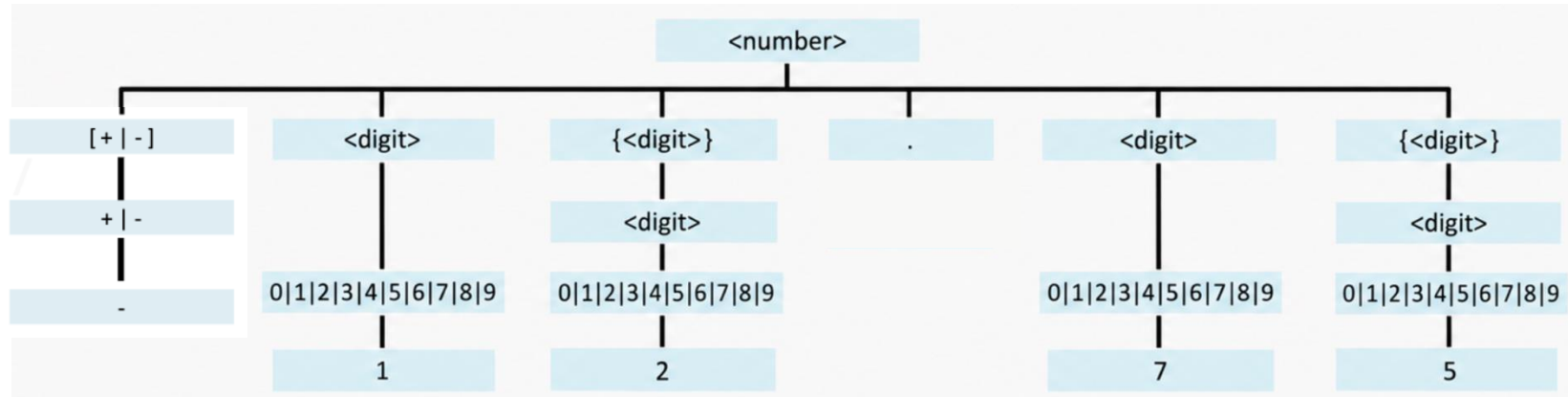
`<digit>` $\leftarrow 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9$

`<number>` $\leftarrow [+ \mid -] \text{ <digit> } \{ \text{<digit>} \} . \text{ <digit> } \{ \text{<digit>} \}$

Beispiel: Ableitung von -12.75 als Baum

<digit> ← 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

<number> ← [+ | -] <digit> {<digit>} . <digit> {<digit>}



Repetition: Ableitungen

- Ableitungsbaum
 - Wurzel ist Namen der Startregel
 - Blätter sind Zeichen
 - Verbindungen stehen für **einen** Ableitungsschritt
- **Ableitungstabelle**
 - Erste Zeile ist Startregel
 - Letzte Zeile ist Zeichenfolge
 - Übergang zwischen zwei Zeilen entspricht **einem** Ableitungsschritt

Repetition: Ableitungen

R1	<digit>	←	0 1 2 3 4 5 6 7 8 9
R2	<integer>	←	[+ -] <digit> {<digit>}

Beispiel: Ableitung von -127 als Tabelle

(R1) <digit> ← 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

(R2) <integer> ← [+ | -] <digit> {<digit>}

<integer> ← [+ | -] <digit> {<digit>}

 ← + | - <digit> {<digit>}

 ← - <digit> {<digit>}

 ← - <digit> <digit> <digit>

 ← - 1 <digit> <digit>

 ← - 1 2 <digit>

 ← - 1 2 7

(R2)

Option gewählt

- gewählt

2 mal wiederholt

(R1) und 1 gewählt

(R1) und 2 gewählt

(R1) und 7 gewählt

Fragen?

Warum EBNF?

EBNF – Einfaches Beispiel – Bahnhof



EBNF – Einfaches Beispiel – Bahnhof



EBNF – Einfaches Beispiel – Bahnhof



EBNF-Beschreibung von Anzeigetafel

<zugbezeichnung> <= IC61 | RE | S3 | EC

<abfahrtszeit> <= <stunde> : <minute>

<zielbahnhof> <= Zürich HB | Bern | Basel | Wil | Chur | Interlaken Ost

<stunde> = ?

<minute> = ?

<anzeigetafel> <= <zugbezeichnung> <abfahrtszeit> <zielbahnhof>

EBNF – Beispiel Programmiersprache

```
<programm> <= PROGRAM <bezeichner>
      BEGIN
      { <zuweisung> ; }
      END
<zuweisung> <= <bezeichner> := ((<zahl>|<bezeichner>)|<string>)
<bezeichner> <= <buchstabe> {<buchstabe>|<ziffer>}
<zahl> <= [-|+] <ziffer> {<ziffer>}
<string> <= "{<buchstabe>|<ziffer>}"
<buchstabe> <= A|B|C|D|E|F|G|H|I|J|K|L|M|N|O|P|Q|R|S|T|U|V|W|X|Y|Z
<ziffer> <= 0|1|2|3|4|5|6|7|8|9
```

<https://docs.oracle.com/javase/specs/jls/se25/html/jls-19.html>

Fragen?

EBNF Notation

- In alten Prüfungen wird oft kursiv verwendet für EBNF-Regeln.
- *digit* statt <digit>
- Ab diesem Semester ist eine EBNF-Regel nur korrekt, wenn sie durch < > gekennzeichnet ist.
- Ebenfalls verwenden wir <- statt <=, beides wird aber als korrekt bewertet.

EBNF: Legal / Nicht Legal

Gegeben sei die EBNF-Beschreibung von *value*

digit $\Leftarrow$ $\boxed{0} \mid \boxed{1} \mid \boxed{2} \mid \boxed{3} \mid \boxed{4} \mid \boxed{5} \mid \boxed{6} \mid \boxed{7} \mid \boxed{8} \mid \boxed{9}$

separator $\Leftarrow$ $\boxed{-}$

buchst $\Leftarrow$ $\boxed{A} \mid \boxed{B} \mid \boxed{C} \mid \boxed{D} \mid \boxed{E} \mid \boxed{F} \mid \boxed{a} \mid \boxed{b} \mid \boxed{c} \mid \boxed{d} \mid \boxed{e} \mid \boxed{f}$

zahl $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [$\boxed{.}$ *digit* { *digit* }]

bd $\Leftarrow$ *buchst* | *digit*

mix1 $\Leftarrow$ *bd* { *bd* }

mix2 $\Leftarrow$ *digit* { *digit* } $\boxed{k}$

mix $\Leftarrow$ *mix1* | *mix2*

value $\Leftarrow$ *zahl* | *real* | *int* | *mix*

2023

EBNF: Legal / Nicht Legal

Gegeben sei die EBNF-Beschreibung von *value*

digit $\Leftarrow$ $\boxed{0} \mid \boxed{1} \mid \boxed{2} \mid \boxed{3} \mid \boxed{4} \mid \boxed{5} \mid \boxed{6} \mid \boxed{7} \mid \boxed{8} \mid \boxed{9}$

separator $\Leftarrow$ $\boxed{-}$

buchst $\Leftarrow$ $\boxed{A} \mid \boxed{B} \mid \boxed{C} \mid \boxed{D} \mid \boxed{E} \mid \boxed{F} \mid \boxed{a} \mid \boxed{b} \mid \boxed{c} \mid \boxed{d} \mid \boxed{e} \mid \boxed{f}$

zahl $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [$\boxed{.}$ *digit* { *digit* }]

bd $\Leftarrow$ *buchst* | *digit*

mix1 $\Leftarrow$ *bd* { *bd* }

mix2 $\Leftarrow$ *digit* { *digit* } $\boxed{k}$

mix $\Leftarrow$ *mix1* | *mix2*

value $\Leftarrow$ *zahl* | *real* | *int* | *mix*

_2023

EBNF: Legal / Nicht Legal

Gegeben sei die EBNF-Beschreibung von *value*

digit $\Leftarrow$ $\boxed{0} \mid \boxed{1} \mid \boxed{2} \mid \boxed{3} \mid \boxed{4} \mid \boxed{5} \mid \boxed{6} \mid \boxed{7} \mid \boxed{8} \mid \boxed{9}$

separator $\Leftarrow$ $\boxed{-}$

buchst $\Leftarrow$ $\boxed{A} \mid \boxed{B} \mid \boxed{C} \mid \boxed{D} \mid \boxed{E} \mid \boxed{F} \mid \boxed{a} \mid \boxed{b} \mid \boxed{c} \mid \boxed{d} \mid \boxed{e} \mid \boxed{f}$

zahl $\Leftarrow$ *digit* { [*separator*] *digit* }

int $\Leftarrow$ *digit* { *digit* }

real $\Leftarrow$ *digit* { *digit* } [$\boxed{.}$ *digit* { *digit* }]

bd $\Leftarrow$ *buchst* | *digit*

mix1 $\Leftarrow$ *bd* { *bd* }

mix2 $\Leftarrow$ *digit* { *digit* } $\boxed{k}$

mix $\Leftarrow$ *mix1* | *mix2*

value $\Leftarrow$ *zahl* | *real* | *int* | *mix*

1a2b3k

Kahoot

Pause

Commit UND PUSH

PULL, dann Push

Häufige Git-Fehler vermeiden

- Vor erster Benutzung auf gitlab.inf.ethz.ch anmelden
- In U02 Aufgabe 4, <nethz-account> im Link inklusive <> ersetzen
- In U02 Aufgabe 4, bei Problemen Link per Hand abtippen
- Files/Directories/Projects/... immer in IntelliJ löschen bzw. ändern (nie im File Explorer/Finder)
- Immer Pull und dann Push (checke Online ob Änderungen sichtbar sind)
- “Commit und Push” statt nur “Commit”
 - Falls Sie auf Commit geklickt haben, drücken Sie manuell per Rechtsklick auf das Repository und wählen Sie Git -> Push aus. Importieren bei neuen Projects nicht vergessen!

KI Funktionen deaktivieren

- Erstellen Sie eine Beschreibung <palindrome>, welche als legale Symbole alle Zahlen zulässt, die von vorne und hinten gleich gelesen werden und die nur die Ziffern von 1 bis 4 verwenden. Beispiele sind 11, 232, 444

-> *siehe TextEdit*

- Erstellen Sie eine Beschreibung `<palindrome>`, welche als legale Symbole alle Zahlen zulässt, die von vorne und hinten gleich gelesen werden und die nur die Ziffern von 1 bis 4 verwenden. Beispiele sind 11, 232, 444

```
<digit> <= 1 | 2 | 3 | 4
```

```
<palindrome> <= <digit>
```

```
| 1 [<palindrome>] 1
```

```
| 2 [<palindrome>] 2
```

```
| 3 [<palindrome>] 3
```

```
| 4 [<palindrome>] 4
```

$\langle \text{digit} \rangle \leq 1 \mid 2 \mid 3 \mid 4$

$\langle \text{palindrome} \rangle \leq \langle \text{digit} \rangle \mid \epsilon$
| 1 $\langle \text{palindrome} \rangle$ 1
| 2 $\langle \text{palindrome} \rangle$ 2
| 3 $\langle \text{palindrome} \rangle$ 3
| 4 $\langle \text{palindrome} \rangle$ 4

$\langle \text{digit} \rangle \leq 1 \mid 2 \mid 3 \mid 4$

$\langle \text{palindrome} \rangle \leq \langle \text{digit} \rangle$
| 1 1 | 2 2 | 3 3 | 4 4
| 1 $\langle \text{palindrome} \rangle$ 1
| 2 $\langle \text{palindrome} \rangle$ 2
| 3 $\langle \text{palindrome} \rangle$ 3
| 4 $\langle \text{palindrome} \rangle$ 4

$\langle \text{digit} \rangle \leq 1 \mid 2 \mid 3 \mid 4$

$\langle \text{palindrome} \rangle \leq \langle \text{digit} \rangle \mid \varepsilon$
| 1 $\langle \text{palindrome} \rangle$ 1
| 2 $\langle \text{palindrome} \rangle$ 2
| 3 $\langle \text{palindrome} \rangle$ 3
| 4 $\langle \text{palindrome} \rangle$ 4

Achtung leeres Wort

- Erstellen Sie eine Beschreibung für <oddEight>, die alle Zahlen enthält, in denen die Ziffer 8 ungerade oft vorkommt.

- Erstellen Sie eine Beschreibung für <oddEight>, die alle Zahlen enthält, in denen die Ziffer 8 ungerade oft vorkommt.

```
<notEight> <= 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 9
```

```
<oddEight> <= {<notEight>} 8 {<notEight>} {8 {<notEight>} 8 {<notEight>}}
```

Prüfungsbeispiele EBNF

Danke!